

User Guide

JALoP over HTTP (v2.x)

29 April 2026

This Page is Intentionally Left Blank

Table of Contents

1	JALoPv2.x.....	6
2	Download/Clone Source Code	6
3	Build and Install C Publisher (jald) and C/C++ Subscribers (jal_subscribe, jal_sub_cpp).....	6
3.1	System Provisioning	6
3.1.1	Install Rust Dependencies	6
3.1.2	RHEL 7 / CentOS 7	7
3.1.3	RHEL 8 / CentOS 8	8
3.1.4	RHEL 9	9
3.1.5	RHEL 10	10
3.2	Build and Install C Publisher (jald) and C/C++ Subscribers (jal_subscribe, jal_sub_cpp).....	10
3.3	Configure C Publisher (jald)	11
3.4	Configure JAL-Local-Store (jal-local-store).....	11
3.5	Configure C/C++ Subscriber (jal_subscribe, jal_sub_cpp)	12
3.6	Configure JALoP Inline Filter	12
3.7	Configure LMDB (optional)	12
4	Build and Install Java Subscriber (jnl_test-2.1.x.x.jar)	12
4.1	System Provisioning	12
4.2	Build and Install Java Subscriber	12
4.3	Configure Java Subscriber (jnl_test)	12
5	Run Publisher and Subscriber to Transfer JAL Records	13
5.1	Disable SELinux (Test Environment Only)	13
5.2	Stop/Disable Firewall (Test Environment Only).....	13
5.3	Start JAL-LOCAL-STORE:	13
5.4	Insert Records into JAL-Local-Store	13
5.4.1	Insert Log Records.....	13
5.4.2	Insert Audit Records.....	14
5.4.3	Insert Journal Records.....	14
5.5	Check for Inserted Records in JAL-Local-Store.....	14
5.6	Start Rust Inline Filter (Recommended).....	14
5.7	Start Publisher.....	14

5.8	Start Java Subscriber (jjnl_test).....	15
5.9	Start C/C++ Subscribers (jal_subscribe, jal_sub_cpp).....	15
5.10	Purge Records from Local Store.....	16
5.11	Run the Publisher (jald) with GDB	16
5.12	Run the Publisher (jald) with Valgrind	16
6	JALoP Data-Taps.....	16
6.1	jalop-coreutils	16
6.1.1	Build & Install	16
6.1.2	Run tee and tail	17
6.2	jalop-jalauditd	17
6.2.1	Build & Install	17
6.2.2	Configure jalauditd.....	18
6.2.3	Run jalauditd	18
6.3	jalop-log4cxx	18
6.3.1	Build & Install	18
6.3.2	Create, Build & Run a Test Executable	19
6.4	jalop-rsyslogd	21
6.4.1	Build & Install	21
6.4.2	Configure omjal plugin	22
6.4.3	Install omjal plugin	22
6.5	jalop-jaljournald	23
6.5.1	Build & Install.....	23
6.5.2	Configure jaljournald	23
6.5.3	Run jaljournald	24
7	Seccomp (Secure Computing).....	25
7.1	JALoP Programs that are seccomp enabled and use JalSeccompEnforcer	25
7.2	JalSeccompEnforcer - api	25
7.3	Positioning of the JalSeccompEnforcer code in JALoP programs	25
7.4	Configuration Settings	25
7.5	How to initialize your rule sets	25
7.6	Test your rule sets.....	26
7.7	When using seccomp_debug = true on RHEL 8, RHEL 9 or RHEL 10.....	26
7.8	When using seccomp_debug = true on RHEL 7	26

7.9 Using the Ansible playbook..... 26

This Page is Intentionally Left Blank

1 JALoPv2.x

JALoPv2.x is JALoP over HTTP. It has a C Publisher, Subscribers (available in C, C++, and Java), and several data-taps. The Publisher usually resides on a CDS system to securely and reliably transfer journal, audit, and log records generated by the CDS to one or more remote Subscribers.

2 Download/Clone Source Code

Create a top-level ‘jalop’ directory to store all JALoP repositories. This will be referred to as <jalop_root> throughout this document.

```
% mkdir jalop
% cd jalop/
```

JALoPv2.x Publisher and Subscriber repositories are available on GitHub at this URL –

<https://github.com/JALoP>

Clone “JALoP” and “jjnl” repositories using git –

```
% git clone https://github.com/JALoP/JALoP.git
% cd JALoP/
% git checkout -t origin/2.x.x.x (if not already on 2.x.x.x branch)

% git clone https://github.com/JALoP/jjnl.git
% cd jjnl/
% git checkout -t origin/2.x.x.x (if not already on 2.x.x.x branch)
```

Initialize Git Large File storage (LFS) for the JALoP git repo

```
% sudo yum install git-lfs
% cd <jalop_root>/JALoP/
% git lfs install
% git lfs env
% git lfs pull
```

3 Build and Install C Publisher (jald) and C/C++ Subscribers (jal_subscribe, jal_sub_cpp)

This is in “jalop” (or “JALoP” if cloned from GitHub) repository. Clone the repository as mentioned above, if not done yet. The C Publisher (jald) is the process that negotiates with the Subscriber(s) and sends JAL records to them. The C/C++ Subscriber(s) provide

3.1 System Provisioning

3.1.1 Install Rust Dependencies

NOTE: This section can be skipped if the pre-built JALoP-rust-deps-*.rpm are present under <jalop_root>/3rd-party/rust/RHEL{7,8,9}.

JALoP uses Rust and requires the following steps to set up the required dependencies. These steps must be run on a RHEL 9 machine/vm first, with the resulting generated dependency tar.gz files copied to RHEL 7, RHEL 8, or RHEL 10 if desired to build on these RHEL versions.

- Install EPEL
 - `$ sudo yum install epel-release # if it is not installed already.`
- Run the following script in <jalop_root>/JALoP to package all of the rust dependencies needed on RHEL 7/8/9/10.
 - `$ cd <jalop_root>/JALoP`
 - `$ sudo sh create_cargo_vendor_files.sh`
- Copy the resulting "vendor-cargo-<rhel version>.tar.gz" file to the <jalop_root>/JALoP dir on the machine building the specific RHEL version of JALoP. Note: This step can be skipped on RHEL 9, as the previous step already generated the RHEL 9 version of the JALoP rust dependencies in <jalop_root>/JALoP.
- Run the following script in <jalop_root>/JALoP
 - `$ cd <jalop_root>/JALoP`
 - `$ sudo sh install_cargo_vendor_files.sh`

3.1.2 RHEL 7 / CentOS 7

The following instructions will allow a CentOS 7 minimal install to be provisioned to build and run the JALoP C implementation:

- Install EPEL
 - `$ sudo yum install epel-release`
- Install JALoP dependencies available from repos
 - `$ sudo yum install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel xmlsec1-openssl-devel python2-scons python36-scons lcov libtool-ltdl-devel libcurl-devel doxygen libseccomp-devel systemd-devel libcap-devel libcap-ng-devel libmicrohttpd-devel texinfo gnutls gnutls-devel lmdb-devel boost-devel clang-devel`
- Install bundled JALoP dependencies from provided RPM
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL7/*.x86_64.rpm`
 - Install JALoP rust dependencies (if present)
 - `$ sudo yum install 3rd-party/rust/RHEL7/*.x86_64.rpm`
- Download and install test-dept unit test library.
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- Download and Install libmicrohttpd library
 - Note that CentOS/RHEL 7 has an available installation of libmicrohttpd, but it too old for our purposes. It is recommended to uninstall the version installed by yum, if present
 - `sudo yum remove libmicrohttpd`
 - `git clone https://git.gnunet.org/libmicrohttpd.git`
 - `git checkout v0.9.59`
 - This is the version used by CentOS/RHEL8, for consistency
 - `./bootstrap`

- `./configure --enable-https --with-gnutls`
 - `make`
 - `sudo make install`
 - See REAMDE for additional configuration options
- NOTE: axl build steps below are only needed if this dependency was not already installed via the provided RPM in the step above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen.sh`
 - `$ sudo make install`
- Enable devtools repository
 - (RHEL Only) `sudo subscription-manager repo --enable rhel-7-server-devtools-rpms`
 - (CentOS Only) `sudo yum install centos-release-scl`
 - `sudo yum install devtoolset-7`
- Ensure `/usr/local/lib` is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.1.3 RHEL 8 / CentOS 8

The following instructions will allow a CentOS 8 Stream minimal install to be provisioned to build and run the JALoP C implementation:

- RHEL 8: Enable CodeReady Builder
 - `$ sudo subscription-manager repos --enable codeready-builder-for-rhel-8-x86_64-rpms`
- CentOS 8: Enable Powertools (unbranded version of RHEL's CodeReady Builder)
 - `$ sudo dnf config-manager --set-enabled powertools`
- Install JALoP dependencies available from repos
 - `$ sudo dnf install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel xmlsec1-openssl-devel libtool-ltdl-devel apr-util-devel libcurl-devel doxygen lcov libseccomp-devel systemd-devel libcap-devel libcap-ng-devel libmicrohttpd-devel lmbd-devel boost-devel clang-devel`
- Install bundled JALoP dependencies from provided RPM
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL8/*.x86_64.rpm`
 - Install JALoP rust dependencies (if present)
 - `$ sudo yum install 3rd-party/rust/RHEL8/*.x86_64.rpm`
- Build and install test-dept
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl build steps below are only needed if this dependency was not already installed via the provided RPM in the step above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`

- `$ cd libaxl`
 - `$ ./autogen --disable-py-axl`
 - `$ sudo make install`
- Install python36-scons
 - `$ sudo yum install python36-scons`
 - `$ sudo pip3 install scons`
- Use python36 as python
 - `$ sudo alternatives --config python`
 - Enter the number for the /usr/bin/python36 selection
- Ensure /usr/local/lib is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.1.4 RHEL 9

The following instructions will allow a RHEL9 minimal install to be provisioned to build and run the JALoP C implementation:

- RHEL 9: Enable CodeReady Builder
 - `$ sudo subscription-manager repos --enable codeready-builder-for-rhel-9-x86_64-rpms`
- Install JALoP dependencies available from repos
 - `$ sudo dnf install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel xmlsec1-openssl-devel libtool-ltdl-devel apr-util-devel libcurl-devel doxygen lcov libseccomp-devel systemd-devel libcap-devel libcap-ng-devel libmicrohttpd-devel lmbd-devel boost-devel clang-devel`
- Install bundled JALoP dependencies from provided RPM
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL9/*.x86_64.rpm`
 - Install JALoP rust dependencies (if present)
 - `$ sudo yum install 3rd-party/rust/RHEL9/*.x86_64.rpm`
- Build and install test-dept
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl build steps below are only needed if this dependency was not already installed via the provided RPM in the step above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen --disable-py-axl`
 - `$ sudo make install`
- Install python3-scons
 - `$ sudo yum install python3-scons`
- Ensure /usr/local/lib is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.1.5 RHEL 10

The following instructions will allow a RHEL10 minimal install to be provisioned to build and run the JALoP C implementation:

- RHEL 10: Enable CodeReady Builder
 - `$ sudo subscription-manager repos --enable codeready-builder-for-rhel-10-x86_64-rpms`
- Install JALoP dependencies available from repos
 - `$ sudo dnf install @development libxml2-devel libconfig-devel libuuid-devel openssl-devel xmlsec1-openssl-devel libtool-ltdl-devel apr-util-devel libcurl-devel doxygen lcov libseccomp-devel systemd-devel libcap-devel libcap-ng-devel libmicrohttpd-devel lmbd-devel boost-devel clang-devel`
- Install bundled JALoP dependencies from provided RPM
 - Navigate into your cloned JALoP repo
 - Install axl RPM
 - `$ sudo yum install 3rd-party/axl/RHEL10/*.x86_64.rpm`
 - Install JALoP rust dependencies (if present)
 - `$ sudo yum install 3rd-party/rust/RHEL10/*.x86_64.rpm`
- Build and install test-dept
 - `$ git clone https://github.com/norrby/test-dept.git`
 - `$ cd test-dept`
 - `$ ./bootstrap`
 - `$ ./configure`
 - `$ sudo make install`
- NOTE: axl build steps below are only needed if this dependency was not already installed via the provided RPM in the step above.
- Build and install axl
 - `$ git clone https://github.com/ASPLes/libaxl.git`
 - `$ cd libaxl`
 - `$ ./autogen --disable-py-axl`
 - `$ sudo make install`
- Install python3-scons
 - `$ sudo yum install python3-scons`
- Ensure /usr/local/lib is in the dynamic linker path
 - `$ echo /usr/local/lib | sudo tee /etc/ld.so.conf.d/usr-local-lib.conf`
 - `$ sudo ldconfig`

3.2 Build and Install C Publisher (jald) and C/C++ Subscribers (jal_subscribe, jal_sub_cpp)

The “jalop” (or “JALoP” if cloned from GitHub) repository has and builds the following applications –

- “jald” (the Publisher)
- “jal-local-store” (the JAL-Local-Store)
- “jaldb_tail”
- “jal_dump”
- “jal_purge”
- “jalp_test” (a test Producer)

- “jal_subscribe” (C Subscriber)
- “jal_sub_cpp” (C++ Subscriber)
- “jaldb_inline_filter” (the JALoP Rust Inline Filter)

It also builds the following shared libraries –

- “libjal-common.so”
- “libjal-db.so”
- “libjal-network.so”
- “libjal-producer.sp”
- “libjal-utils.so”

Follow the instructions below to build and install the above mentioned components -

- Change to the jalop top directory –
 - `% cd <jalop_root>/jalop/` (or, `cd <jalop_root>/JALoP/`, if cloned from github).
- Checkout the v2.x branch of jalop -
 - `% git checkout -t origin/2.x.x.x`
- Clean up jalop –
 - Note: PKG_CONFIG_PATH may need to be set to include `/usr/local/lib/pkgconfig` until we remove vortex dependency in JALoP v2.x.
 - One CentOS/RHEL7, the libmicrohttpd library built from source will also require the PKG_CONFIG_PATH to be set to wherever the .pc file is placed, which is also `/usr/local/lib/pkgconfig` by default
 - (CentOS7/RHEL7 Only) `scl enable devtoolset-7 $SHELL`
 - This enables a fully C++17 compliant version of gcc
 - `% export PKG_CONFIG_PATH="/usr/local/lib/pkgconfig"`
 - `% scons -c`
- Build jalop and install publisher and subscribers using the LMDB database
 - `% scons`
 - `% sudo ./install_rhel_x86_64.sh`

3.3 Configure C Publisher (jald)

Make a copy of `<jalop_root>/jalop/test-input/jald.cfg` and adjust according to your Subscriber’s IP, port, etc. This is to avoid changing the sample configuration file in the git source tree.

3.4 Configure JAL-Local-Store (jal-local-store)

The JAL-Local-Store (`jal-local-store`) is a process that receives and stores JAL Data sent from the JAL Producer applications. It has a Lightning Memory-Mapped Database (LMDB) to store and process the JAL records. The `jal-local-store` process must be started before the Publisher process (`jald`).

Make a copy of `<jalop_root>/jalop/test-input/local_store.cfg` and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

3.5 Configure C/C++ Subscriber (jal_subscribe, jal_sub_cpp)

Make a copy of <jalop_root>/jalop/test-input/jal_subscribe.cfg and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

3.6 Configure JALoP Inline Filter

Make a copy of <jalop_root>/jalop/test-input/jaldb_inline_filter.toml and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

3.7 Configure LMDB (optional)

Optionally, a file named LMDB_CONFIG can be placed at the <db_root> directory to override LMDB specific configuration settings. The two configuration settings that can be overridden are database_option and lmdb_map_size. If the LMDB_CONFIG file is not present and/or does not contain these settings, then default values are used. For lmdb_map_size, which sets the maximum allowed size of the lmdb database, the default is 120 gigabytes. For database_option, the default is LMDB performance level 2 (JDB_LMDB_PERFORMANCE_LEVEL2). These settings are global across all JAL applications.

Make a copy of <jalop_root>/jalop/test-input/LMDB_CONFIG into the db root directory (which is the directory specified by the db_root config setting in the local store configuration file) and update accordingly.

4 Build and Install Java Subscriber (jnl_test-2.1.x.x.jar)

This is the “jjnl” repository. Clone the repository as below (if not done yet) –

```
% cd <jalop_root>/
% git clone https://github.com/JALoP/jjnl.git
% cd jjnl
% git checkout -t origin/2.x.x.x (if not already in that branch)
```

4.1 System Provisioning

Install the following packages –

```
% sudo yum install java-11-openjdk-devel ant maven
```

4.2 Build and Install Java Subscriber

- Build the “master” branch for JALoPv2.x -
 - % cd <jalop_root>/jjnl/jnl_parent/
 - % mvn clean
 - % mvn -Djava.version=1.8 -U clean package (for Java 8 build), or
 - % mvn -Djava.version=11 -U clean package (for Java 11 build)
- Install (optional) -
 - % cd <jalop_root>/jjnl/jnl_lib/
 - % mvn install

4.3 Configure Java Subscriber (jnl_test)

```
% cd <jalop_root>/jjnl/jnl_test/
```

Copy `./jnl_test/target/test-classes/sampleHttpSubscriber.json` here and update accordingly. This is to avoid changing the sample config file in git source tree.

Check section “Run the Publisher and Subscriber to Transfer JAL Records” below.

5 Run Publisher and Subscriber to Transfer JAL Records

Follow the steps below to run the C Publisher and Java Subscriber to transfer JAL records.

5.1 Disable SELinux (Test Environment Only)

```
=====
Permanent: have "SELINUX=disabled" in "/etc/selinux/config" file,
restart VM.
Temporary: Enter the command "/usr/sbin/setenforce 0"
```

5.2 Stop/Disable Firewall (Test Environment Only)

```
=====
RHEL/CentOS 7.x and higher:
                % sudo service firewalld stop
Or,             % sudo systemctl stop firewalld
Disable: % sudo systemctl disable firewalld
```

5.3 Start JAL-LOCAL-STORE:

```
=====
# May need to clean up the first time. Make sure no jal-local-store is
running.
$ cd <jalop_root>/jalop/
$ pkill jal-local-store
$ sudo rm -rf ./jal.sock
$ sudo rm -rf /root/testdb
$ sudo mkdir /root/testdb
```

Make a copy of `./test-input/local_store.cfg` here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ sudo ./release/bin/jal-local-store --debug --no-daemon -c
./local_store.cfg &
```

For 2.0.0.2-beta and older:

```
$ sudo ./release/bin/jal-local-store --debug ./local_store.cfg &
```

5.4 Insert Records into JAL-Local-Store

5.4.1 Insert Log Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop-test-
data/input/app_meta/log4cxx-warn.cfg -p ~/jalop/jalop-test-
data/input/journal/big_payload.txt -n 100 -t 1
```

5.4.2 Insert Audit Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop-test-
data/input/app_meta/log4cxx-warn.cfg -p ~/jalop/jalop-test-
data/input/journal/big_payload.txt -n 100 -t a
```

5.4.3 Insert Journal Records

```
=====
$ sudo ./release/bin/jalp_test -j ./jal.sock -a ~/jalop/jalop-test-
data/input/app_meta/log4cxx-warn.cfg -p ~/jalop/jalop-test-
data/input/journal/big_payload.txt -n 100 -t j
```

Note: Use "-t f" instead of "-t j" if you want JAL Producer to send journal file descriptor through the Unix Domain Socket (instead of the payload bytes) to JAL-Local-Store. Passing file descriptor may perform better for larger journal payloads (e.g., in the order of megabytes).

5.5 Check for Inserted Records in JAL-Local-Store

```
=====
$ sudo ./release/bin/jaldb_tail -n 100 -h /root/testdb/ -t l | wc -l
$ sudo ./release/bin/jaldb_tail -n 100 -h /root/testdb/ -t a | wc -l
$ sudo ./release/bin/jaldb_tail -n 100 -h /root/testdb/ -t j | wc -l
```

Or,

```
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2033-11-11T11:11:11
-x -t l | wc -l
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2033-11-11T11:11:11
-x -t a | wc -l
$ sudo ./release/bin/jal_purge -h /root/testdb -b 2033-11-11T11:11:11
-x -t j | wc -l
```

5.6 Start Rust Inline Filter (Recommended)

```
=====
$ cd <jalop_root>/jalop/
```

NOTE: If verbose logging is desired optionally run the following command below.

```
$ export RUST_LOG=trace
```

```
$ sudo rm -f jald_filter_socket_*
```

```
$ sudo ./release/bin/jaldb_inline_filter -c test-input/jaldb-
filter.toml
```

Enter "./release/bin/jaldb_inline_filter -h" for help.

5.7 Start Publisher

```
=====
```

```
$ cd <jalop_root>/jalop/
```

Make a copy of ./test-input/jald.cfg here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ sudo ./release/bin/jald -f -d -c ./jald.cfg --no-daemon 2>&1 | tee publisher.log
```

NOTE: To run jald without using the inline filter use the following command below instead (no -f option). It is not recommended to run jald publisher without the inline filter.

```
$ sudo ./release/bin/jald -d -c ./jald.cfg --no-daemon 2>&1 | tee publisher.log
```

Enter "man jald" for help.

5.8 Start Java Subscriber (jjnl_test)

=====

```
$ cd <jalop_root>/jjnl/jnl_test/
```

Make a copy of ./jnl_test/target/test-classes/sampleHttpSubscriber.json here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

```
$ java -jar target/jnl_test-2.0.0.0.jar ./sampleHttpSubscriber.json 2>&1 | tee subscriber.log
```

5.9 Start C/C++ Subscribers (jal_subscribe, jal_sub_cpp)

=====

```
$ cd <jalop_root>/jalop/
```

Make a copy of ./test-input/jal_subscribe.cfg here and update accordingly. This is to avoid changing the sample configuration file in the git source tree.

To run the installed binary against installed libraries, use the following:

```
$ [jal_subscribe|jal_sub_cpp] -c jal_subscribe.cfg 2>&1 | tee subscriber.log
```

To instead run a locally built binary against locally built libraries (not suitable for deployment to real hardware):

```
$ export LD_LIBRARY_PATH=<repo_root>/[debug|release]/lib/  
$ ./[debug|release]/bin/[jal_subscribe|jal_sub_cpp] -c jal_subscribe.cfg 2>&1 | tee subscriber.log
```


5.10 Purge Records from Local Store

```
=====
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2033-11-
11T11:11:11 -t l
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2033-11-
11T11:11:11 -t a
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b 2033-11-
11T11:11:11 -t j
```

OR,

```
$ now=$(date -u "+%Y-%m-%dT%H:%M:%S.%N")
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t l
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t a
$ sudo ./release/bin/jal_purge -h ./testdb/ -d -f -c -b "$now" -t j
```

5.11 Run the Publisher (jald) with GDB

```
=====
$ export LD_LIBRARY_PATH="/home/marefin/jalop/jalop/debug/lib/"
$ gdb -ex=r --args ./debug/bin/jald -d -c ./jald.cfg --no-daemon
```

5.12 Run the Publisher (jald) with Valgrind

```
=====
$ valgrind --tool=memcheck --leak-check=full --verbose --track-
origins=yes --log-file=valgrind_out.txt ./debug/bin/jald -d -c
./jald.cfg --no-daemon
```

6 JALoP Data-Taps

The JALoP data-taps capture and send JAL records to the JALoP Local Store (jal-local-store). Each data-tap depends on JALoP libraries, so JALoP v1 or v2 libraries must be build and installed prior to building a data-tap repository. Generally, each of the data-taps requires the JALoP socket and db_root addresses that jal-local-store (JLS) uses. Note that jal-local-store must be already running for any data-tap to send JAL records to it via the socket.

6.1 jalop-coreutils

Available on IntelLink and JALoP development Git Server.

This has the JALoP version of GNU tail and GNU tee commands.

6.1.1 Build & Install

```
% cd <jalop_root>/jalop-coreutils/
% autoreconf -fiv # for RHEL7 only
% ./bootstrap --skip-po # for RHEL8/RHEL9/RHEL10 only
% ./configure --disable-gcc-warnings
% make -j
```

```
% make check
% [[ -e /tmp/install ]] || mkdir /tmp/install
% make install DESTDIR=/tmp/install
% cp /tmp/install/usr/local/bin/{tee,tail} /usr/local/bin
```

6.1.2 Run tee and tail

Note that the jal-local-store (JLS) process must be running already to receive and insert records.

```
# Based on <db_root> location, user may need 'sudo' in the
following instructions.

% echo "This is a test message to tee into JLS" | tee -j --
path=<path to JALoP socket> --schema-root=<path to JALoP schema
directory>
% echo -e "Test message1\nTest message2" > file_to_tail
% tail -j --path=<path to JALoP socket> --schema-root=<path to
JALoP schema directory> ./file_to_tail
```

Use `jaldb_tail` to check for newly inserted records -

```
% jaldb_tail -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from auditd -

```
% jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t 1 -d
z
```

6.2 jalop-jalauditd

Available on the NCDSMO Intelink and JALoP development Git Server. This is also available at [GitHub.com/JALoP/JALoP-Auditd-Plugin](https://github.com/JALoP/JALoP-Auditd-Plugin).

6.2.1 Build & Install

```
% cd <jalop_root>/jalop-jalauditd (or JALoP-Auditd-Plugin)
% make clean
% make
% sudo make install
```

This shall install the followings –

- The binary `/sbin/jalauditd` (or `/usr/sbin/jalauditd`)
- The configuration files for RHEL7 and RHEL8/RHEL9/RHEL10 go into different directories. make install will install the configuration files in the following locations and will need to be copied to appropriate directory:
 - `/etc/jalauditd/jalauditd.conf`
 - `/etc/audit/plugins.d/audisp-jalauditd.conf`
- For RHEL7 the configuration files should be:
 - `/etc/audisp/jalauditd.conf`
 - `/etc/audisp/plugins.d/audisp-jalauditd.conf`
- For RHEL8/RHEL9/RHEL10 the configuration files should be:

- `/etc/jalauditd/jalauditd.conf`
 - `/etc/audit/plugins.d/audisp-jalauditd.conf`
- The `jalauditd.conf` file is initially empty; you will need to edit this file, see below.

6.2.2 Configure jalauditd

The configuration file `jalauditd.conf` is initially empty. There can be 4 settings in this file as shown below.

```
socket = "/path/to/jalop/socket";
schemas = "/path/to/schemas/root";
keypath = "/path/to/key";
certpath = "/path/to/cert";
```

If the `socket` and `schemas` locations are not specified above, default locations specified by the JAL Producer Library (JPL) will be used. Below are the default socket and schemas locations –

```
socket = "/var/run/jalop/jalop.sock"
schemas = "/usr/share/jalop/schemas"
```

If `keypath` or `certpath` are not specified, no key or cert will be used for signing.

IMPORTANT: These settings must be consistent with the `jal-local-store` configuration file.

6.2.3 Run jalauditd

Note that the `jal-local-store` (JLS) process must run to receive and insert records sent by `jalauditd` via the socket.

Restarting `auditd` automatically runs the `jalauditd` child process.

```
% service auditd restart (or, systemctl restart auditd)
```

Use `jaldb_tail` to check for newly inserted records -

```
% sudo jaldb_tail -f -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from `auditd` -

```
% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t
1 -d z
```

See `<jalop_root>/jalop-jalauditd/README` file for more details.

6.3 jalop-log4cxx

Available on IntelLink and JALoP development Git Server.

6.3.1 Build & Install

Note that JALoP must be built and installed before attempting to build the `jalop-log4cxx` library.

```
$ sudo yum install apr-util-devel
```

On RHEL 7 / CentOS 7:

```
$ sudo yum install cmake3
$ sudo yum install centos-release-scl
$ sudo yum install devtoolset-8-gcc-c++ --
enablerepo='centos-sclo-rh'
$ sudo scl enable devtoolset-8 'bash'
```

On RHEL 8 / CentOS 8 / RHEL9 / RHEL10:

```
$ sudo yum install cmake
```

Build the library:

On RHEL 7 / CentOS 7:

```
$ cd jalop-log4cxx
$ cmake3 -D CMAKE_CXX_COMPILER=/opt/rh/devtoolset-
8/root/usr/bin/g++ -D CMAKE_INSTALL_PREFIX=/usr CMakeLists.txt
$ make
```

On RHEL 8 / CentOS 8 / RHEL9 / RHEL10:

```
$ cd jalop-log4cxx
$ cmake -D CMAKE_INSTALL_PREFIX=/usr CMakeLists.txt
$ make
```

Install the library:

```
$ sudo make install
```

To uninstall the library:

```
$ sudo make uninstall
```

6.3.2 Create, Build & Run a Test Executable

An example test executable is created during the CMake build at `/path/to/jalop-log4cxx/src/examples/cpp/jalp-appender`. To test using this executable, skip ahead to the instructions on starting the `jal_local_store` and running the executable.

To create a new test executable, do the following:

Create a separate folder for the test executable. This can be separate from the `jalop-log4cxx` folder:

```
$ mkdir ~/log4cxx-test
$ cd ~/log4cxx-test
```

Copy required files from `jalop-log4cxx`:

```
$ cp /path/to/jalop-log4cxx/src/main/cpp/jalpappender.cpp .
```

Create a main.cpp for the text executable. Use the following as an example:

```
#include <log4cxx/logger.h>
#include <log4cxx/propertyconfigurator.h>
#include <log4cxx/jalpappender.h>

using namespace log4cxx;

LoggerPtr logger(Logger::getLogger("Test"));

int main(int argc, char **argv)
{
    PropertyConfigurator::configure(argv[1]);
    LOG4CXX_INFO(logger, "qwerty")
    return 0;
}
```

Build the test executable:

```
$ g++ main.cpp jalpappender.cpp -llog4cxx -ljavalop-producer -o
log4cxx-test
```

Create a configuration file for the test executable. Use the following as a starting point:

```
log4j.rootLogger=DEBUG,A1

log4j.appender.A1=org.apache.log4j.JalpAppender

log4j.appender.A1.layout=org.apache.log4j.PatternLayout
log4j.appender.A1.layout.ConversionPattern=%-4r [%t] %-5p %c %x -
%m%n
```

The following are optional parameters than can be added to the log4cxx config file:

After starting the **jal-local-store**, run the test executable:

```
$ ./log4cxx-test path/to/test.cfg
```

This will send one record to jal-local-store before closing the connection. Verify the record with the following:

```
$ jaldb_tail -h /path/to/testdb -t 1 -d a
```

```
log4j.appender.A1.PATH - The path to jalop.sock
log4j.appender.A1.HOSTNAME - The hostname of the machine that the
JALoP local store is on
log4j.appender.A1.APPNAME - The name of the process that is logging
data
```

```
log4j.appender.A1.SCHEMA_ROOT - The path to the JALoP schemas
log4j.appender.A1.KEY_PATH - The path to the RSA key used for signing
log4j.appender.A1.CERT_PATH - The path to the Certificate used for
signing
```

This will print the app metadata for the records. The record inserted by the test executable does not create a payload from the string in LOG4CXX_INFO. It ends up in the message tag in the app metadata.

6.4 jalop-rsyslogd

Available on IntelLink and JALoP development Git Server.

6.4.1 Build & Install

Note that JALoP must be built and installed before attempting to build the rsyslog plugin.

```
$ sudo yum install git automake libtool libestr-devel
```

If you have access to a libfastjson-devel package, you can install that and skip to Build omjal plugin:

```
$ sudo yum install libfastjson-devel
```

6.4.1.1 Libfastjson headers

If you cannot install libfastjson-devel, the libfastjson headers must be made available to the rsyslog plugin in another way.

```
$ git clone https://github.com/rsyslog/libfastjson.git
$ cd libfastjson
$ sudo mkdir /usr/include/libfastjson
$ sudo cp ./*.h /usr/include/libfastjson
$ sudo ln -s /usr/lib64/libfastjson.so.4
/usr/lib64/libfastjson.so
```

On RHEL 7 / CentOS 7:

```
$ export JSON_C_CFLAGS="-g"
$ export JSON_C_LIBS="/usr/lib64/libfastjson.so"
```

On RHEL 8 / CentOS 8:

```
$ export LIBFASTJSON_CFLAGS="-g"
$ export LIBFASTJSON_LIBS="/usr/lib64/libfastjson.so"
```

6.4.1.2 Build omjal plugin

Build the omjal plugin:

```
$ cd jalop-rsyslogd
$ export CPPFLAGS="-I/usr/include/libfastjson"
$ sh autogen.sh
$ make
```

6.4.2 Configure omjal plugin

Add the following configuration section to `/etc/rsyslog.conf`:

```
$ModLoad omjal # calls JAL for rsyslog pass through to JALoP
$ActionOmjalSocket /var/run/jalop/jalop.sock
$ActionOmjalSchemas /usr/share/jalop/schemas
$ActionOmjalKey <path_to_key> # Optional
$ActionOmjalCert <path_to_key> # Optional
*. * :omjal:
```

`$ModLoad omjal`

States that the omjal plugin shall be loaded

`$ActionOmjalSocket /var/run/jalop/jalop.sock`

Specifies the socket to be used to forward the messages to the JALoP local store. If this option is omitted from the file, it will use the default value specified in the JALoP library.

`$ActionOmjalSchemas /usr/share/jalop/schemas`

Specifies the directory where the schema information is located. If this option is omitted from the file, it will use the default value specified in the JALoP library.

`$ActionOmjalKey <path_to_key>`

Specifies the path to the key to be used for signing. If this option is omitted from the file, then no key will be used.

`$ActionOmjalCert <path_to_key>`

Specifies the path to the certificate to be used for signing. If this option is omitted from the file, then no certificate will be used.

`*. * :omjal:`

`<msg_filter_expression>:<output_module>:<parameters>`

`*. *` specifies that all messages will be sent to this module. If, for example, you wanted to handle mail messages, `mail.*` would handle that.

See <https://www.rsyslog.com/doc/v8-stable/configuration/filters.html> for more information.

6.4.3 Install omjal plugin

Install the omjal plugin:

```
$ cd jalop-rsyslogd
$ make install
```

The rsyslog package installs libraries to `/usr/lib64/rsyslog`. If your environment uses a different location, either manually copy the library from `.libs/omjal.so` or change the `libdir` flag in `autogen.sh`

After installation, and after starting the *jal-local-store*, restart the rsyslog service:

```
$ sudo systemctl restart rsyslog
```

To ensure that rsyslogd is connecting to jal-local-store, you can run jal-local-store with the debug (-d) flag. When jal-local-store is ready, the message `Thread_count: 1` will appear. After rsyslogd is connected, the message `Thread_count: 2` will appear.

To create messages in rsyslogd, the logger command can be used. For example:

```
logger "This is your message content."
```

Use `jaldb_tail` to check for newly inserted records -

```
% sudo jaldb_tail -f -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from auditd -

```
% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t  
1 -d z
```

6.5 jalop-jaljournal

Available on Intelink and JALoP development Git Server. This is also available at GitHub/JALoP/JALoP-Journal.

6.5.1 Build & Install

Dependencies:

```
/usr/lib64/<jalop shared object files>
```

```
/usr/include/<jalop headers>
```

```
systemd-devel rpm
```

```
% cd <jalop_root>/jalop-jaljournal
```

```
% make clean
```

```
% make
```

```
% sudo make install
```

This shall install the followings –

- The binary `/bin/jaljournal`
- The `jaljournal` configuration file `/etc/jalop/jaljournal.cfg`. This file initially has some default values; you will need to edit this file, see below.

6.5.2 Configure jaljournal

The `jalauditd` configuration file `/etc/jalauditd/jalauditd.conf` initially has some default values; The settings in this file as shown below.

```
# verbose logging  
debug = false;
```



```

# Start collecting logs from head, boot, or tail. Default: tail
# head: will get all journal logs available from multiple boots
#       and then continuously poll to get all logs as they arrive
# boot: will get all logs from current boot and then continuously
#       poll to get all logs as they arrive.
# tail: will continuously poll to get all logs as they arrive.
init_point = "tail";

# collect all journal fields. Default: MESSAGE only
metadata = false;

# socket path that jal-local-store will be listening.
socket = "/path/to/jalop/socket";

# jalop schema path
schemas = "/path/to/schemas/root";

# hostname to record in log
hostname = "localhost";

# appname to record in log
appname = "jalop-journald";

# poll delay between checking for new journal logs
# in milliseconds 1000 = 1 second
delay = 1000;

```

If the `socket` and `schema_path` locations are not specified above, default locations specified by the JAL Producer Library (JPL) will be used. Below are the default `socket` and `schema_path` locations –

```

socket = "/var/run/jalop/jalop.sock"
schema_path = "/usr/share/jalop/schemas"

```

IMPORTANT: These settings must be consistent with the `jal-local-store` configuration file.

6.5.3 Run `jaljournald`

Note that the `jal-local-store` (JLS) process must run to receive and insert records sent by `jaljournald` via the socket. Also, make sure the `jal-local-store` socket path matches the `jaljournald` socket path and you have write permissions on that socket path.

Example command-line

```
% jaljournald -c <path-to-config> --debug --metadata
```

Use `jaldb_tail` to check for newly inserted records -

```
% sudo jaldb_tail -f -t 1 -h <db_root>
```

Use `jal_dump` to verify that the log records came from `journald` -

```
% sudo jal_dump -u <UUID found using jaldb_tail> -h <db_root> -t 1  
-d z
```

7 Seccomp (Secure Computing)

Seccomp is a security feature in the Linux kernel that allows a process to transition into a secure state where it can only make a limited set of system calls. This feature is useful for sandboxing applications, limiting their ability to interact with the system and reducing the attack surface.

7.1 JALoP Programs that are seccomp enabled and use JalSeccompEnforcer

- jal-localstore
- jald
- jal_subscribe

7.2 JalSeccompEnforcer - api

- 1) #include "jal_seccomp_enforcer.h"
- 2) struct jal_seccomp_enforcer_t* seccomp_enforcer =
jal_seccomp_enforcer_create(config_path) ;
- 3) jal_seccomp_enforcer_apply_initial(seccomp_enforcer);
- 4) jal_seccomp_enforcer_apply_final(seccomp_enforcer);

7.3 Positioning of the JalSeccompEnforcer code in JALoP programs

- 1) A JALoP program will use the #include.
- 2) Calls jal_seccomp_enforcer_create(), right after knowing the location of the config file.
- 3) Applies initial, right after creation of the seccomp_enforcer.
- 4) Applies final, right after the program has finished setting up and right before the program starts to do its routine work.

7.4 Configuration Settings

The following configuration settings are available in an applicable JALoP program configuration file under the seccomp setting.

- enable = true;
- debug = false;
- initial = ["arch_prctl", "bind", ...];
- both = ["brk", "ftruncate", ...];
- final = ["accept", "access", ...];

All programs listed above have default seccomp configurations. See various sections above for complete rulesets used sample configuration files.

7.5 How to initialize your rule sets

Do the following in the applicable configuration file under the seccomp setting.

- Set enable = true;
- Set debug = true;
- Comment out any rulesets.

- Run your process with strace and try to have it exercise all its functionality
 - `strace -f -o strace.out <process_name> <process_option>`
- Run the `collect_system_calls.py` script in `<jalop_home>/src/test_utils/scripts/`
 - `collect_system_calls.py strace.out > seccomp.cfg`
- The script above will produce your rule sets.
 - `initial = ["arch_prctl", "bind", ...];`
 - `both = ["brk", "ftruncate", ...];`
 - `final = ["accept", "access", ...];`

7.6 Test your rule sets

- Run your program with `enable_seccomp = true;` and `seccomp_debug = false;`
- If the program runs to completion, that is great.
- If the process stops, the syscall that it failed on will be logged to stdout.
- Add that syscall to the initial set first and run a test again.
- If it fails again for the same syscall, move the syscall to your final set.
- If it fails again for the same syscall, move the syscall to your both set.

7.7 When using `seccomp_debug = true` on RHEL 8, RHEL 9 or RHEL 10

- You need to also have `enable_seccomp = true;`
- There needs to be at least one rule in each set.
- Your process will run freely. Seccomp will not be enforced.
- There will be a log entry in `/var/log/audit/audit.log` for each syscall hit that is not in your rule sets.
- You can `grep` for `SYSCALL` in `/var/log/audit/audit.log`.
- The process will also create a marker syscall while working. This mark is produced at the time `jal_seccomp_enforcer_apply_final(seccomp_enforcer)` is called. The syscall can be identified in the logs by looking for "Applying final ruleset DONE".
- The mark is used by `collect_system_calls.py` to determine what calls should be placed in your initial, both and final rulesets.
- Stderr will log any syscalls in rule sets that are not valid.

7.8 When using `seccomp_debug = true` on RHEL 7

- It will do as above except
 - There is not a `SCMP_ACT_LOG` action in the `libseccomp` library, so no syscalls will be logged to `/var/log/audit/audit.log`.
- You need to run `strace`. (See above in section 8.5 How to initial your rulesets.)

7.9 Using the Ansible playbook

- There is an ansible playbook in `<jalop_home>/src/test_utils/ansible/seccomp.yml`.
- The playbook should already be configured properly as is, but inspect `seccomp.yml` and make any adjustments if needed.
- The sample configuration files used in the playbook should already be configured properly, but inspect your configuration files used and make any adjustments if needed.
- The playbook will run a thorough test of the JALoP processes.

- The playbook will also build your seccomp configurations.
- All output is produced in the <jalop_home>/test_logs directory.
- `ansible-playbook <jalop_home>/src/test_utils/ansible/seccomp.yml -e cwd=`pwd` -e mode=release -e perf=p2 -e rust_log=release -e marker=rust -e rcnt=5000 -e types=jalf`